Laboratorio No. 11

Samuel Robledo, carné 241282
Universidad del Valle de Guatemala, Teoría de la Computación, Laboratorio No. 11

1. Zelkova: SMT para un problema que en general es indecidible

Zelkova es un servicio interno de Amazon que analiza políticas de IAM, de S3 y de otros recursos [1]. La pregunta que responde es si existe *alguna* petición que la política permita y no debería. Como el espacio de peticiones es infinito (cuentas, IP, cadenas en un ARN), probar casos no da una respuesta completa: según el artículo, otras técnicas adivinan y prueban, mientras que Zelkova *sabe* [1].

Para un programa arbitrario que decide accesos, esa pregunta choca con el teorema de Rice: toda propiedad no trivial del comportamiento de un programa es indecidible [17]. Zelkova evita el problema porque no trata la política como un programa, sino que la traduce a una fórmula en un lenguaje restringido (cadenas, expresiones regulares, números, fechas, IP) que un solver SMT sí resuelve. Por ejemplo, una declaración con `NotPrincipal` se vuelve

$$(\mathit{Resource} = \texttt{"arn:aws:s3:::test-bucket"}) \wedge (\mathit{Principal} \neq 111122223333),$$

que es verdadera exactamente cuando la política permite la petición [1]. El solver devuelve una prueba para todos los valores de las variables o un contraejemplo. En mi lectura, el SMT no "resuelve lo indecidible": el problema se reformula dentro de un fragmento lógico que sí es decidible, porque el lenguaje de políticas es mucho menos expresivo que un lenguaje de programación.

2. Video

La charla es *AWS re:Invent 2016: Automated Formal Reasoning About AWS Systems (SEC401)*, del canal Amazon Web Services, presentada por Byron Cook [2]. El artículo del mismo expositor en CAV 2018 desarrolla lo que expone en la charla [3]. Ahí el razonamiento formal automatizado consiste en demostrar propiedades con probadores de teoremas y verificadores de modelos, y Cook afirma que da más garantías que el testing porque deja claro qué quedó asegurado y qué no; además se puede aplicar antes de escribir el código, demostrando los algoritmos y protocolos de alto nivel [3]. Uno de los ejemplos es una herramienta basada en SMT que razona sobre la semántica de las políticas para encontrar las que están mal configuradas, que es lo que hace Zelkova [3, 1].

Cook separa una demostración en dos actividades: buscar una prueba candidata y verificarla. La búsqueda es "el arte" y suele combinar heurísticas que intentan rodear lo indecidible; la verificación normalmente es decidible y lineal en el tamaño de la prueba [3]. Esto conecta con el curso: el problema general no tiene algoritmo que siempre termine, pero comprobar una prueba dada sí, igual que en NP verificar un certificado es fácil aunque encontrarlo no lo sea.

3. SMT

a. ¿Qué es un SMT?

SMT (*Satisfiability Modulo Theories*) es el problema de decidir si una fórmula es satisfacible cuando algunos símbolos tienen un significado fijo dado por una teoría de fondo: aritmética entera o real, bit-vectors, arreglos, cadenas. Extiende SAT, donde todo es booleano: en SMT un átomo puede ser $x + y > 3$. Un enfoque eficiente es DPLL(T): un motor DPLL resuelve la parte booleana y coopera con un solver especializado para la teoría T [4].

Relación con el curso: SAT es NP-completo por el teorema de Cook-Levin [17], así que SMT es al menos igual de difícil. Algunas teorías son indecidibles: para la aritmética entera no lineal no hay procedimiento que siempre termine con `sat` o `unsat`, y `check-sat` puede dar `unknown` o no terminar [5]; con cuantificadores tampoco hay procedimiento de decisión general [6]. En cambio, la aritmética lineal sí se decide (simplex dual para reales; cortes y *branch and bound* para enteros) [5]. Por eso los solvers trabajan sobre fragmentos decidibles y usan heurísticas para el resto.

b. ¿Cómo usa AWS los SMT para analizar políticas?

Zelkova traduce cada política a una fórmula y prueba teoremas sobre ella con solvers SMT [1]. Hace dos preguntas:

- **¿P_1 es menos permisiva que P_2?** Si $P_1 \Rightarrow P_2$ es válida y $P_2 \Rightarrow P_1$ no, P_1 es estrictamente menos permisiva.
- **¿Permite acceso público?** Compara la política con una que deja entrar a usuarios anónimos. Si es más permisiva que esa, hay acceso público; condiciones como `aws:SourceIp` o `aws:SourceVpce` lo impiden.

Según [1], lo usan IAM Access Analyzer for S3, reglas administradas de AWS Config (como `s3-bucket-public-read-prohibited`), Amazon Macie y Amazon GuardDuty.

c. Otros usos de SMT solvers en la industria

1. **SAGE (Microsoft), fuzzing de caja blanca.** Registra las condiciones de las ramas que toma una ejecución, niega una por una y le pide a Z3 una entrada que siga otro camino. Encontró cerca de un tercio de los bugs hallados por *file fuzzing* en el desarrollo de Windows 7 [10].
2. **KLEE, generación automática de pruebas.** Ejecuta con entradas simbólicas y usa el solver STP para decidir qué ramas son posibles y generar casos concretos. Alcanzó más de 90 % de cobertura promedio en las 89 utilidades de GNU Coreutils y encontró 56 bugs serios en 452 aplicaciones [11].
3. **SLAM / Static Driver Verifier (Microsoft).** SDV analiza el código de drivers en modo kernel de Windows y Microsoft lo usó con los drivers del sistema [12]. Su motor es SLAM, y SLAM2 evalúa predicados sobre punteros con Z3 [13, 14].

4. Programación funcional

a. ¿Qué es la programación funcional?

Es un paradigma donde un programa se arma componiendo funciones matemáticas: la salida depende solo de la entrada, sin efectos secundarios ni estado mutable. Se escriben expresiones que se evalúan en vez de instrucciones que modifican variables; las funciones son valores de primera clase, se usa recursión en lugar de ciclos y funciones de orden superior como `map` y `fold` [15].

b. Relación con el cálculo lambda y las máquinas de Turing

El cálculo lambda es su base teórica: solo hay variables, abstracciones $\lambda x.\, M$ y aplicaciones $M\, N$, y el paso de cómputo central es la reducción beta, $(\lambda x.\, M)\, N \to_\beta M[x := N]$ [15]. Un lenguaje funcional es, en el fondo, cálculo lambda con tipos y azúcar sintáctica.

Ejemplo con numerales de Church, donde $\underline{n} = \lambda f.\, \lambda x.\, f^{\,n}(x)$ y $succ = \lambda n.\, \lambda f.\, \lambda x.\, f\,(n\, f\, x)$:

$$\begin{aligned} succ\ \underline{1} &= (\lambda n.\, \lambda f.\, \lambda x.\, f\,(n\, f\, x))\,(\lambda g.\, \lambda y.\, g\, y) \\ &\to_\beta \lambda f.\, \lambda x.\, f\,((\lambda g.\, \lambda y.\, g\, y)\, f\, x) \\ &\to_\beta \lambda f.\, \lambda x.\, f\,((\lambda y.\, f\, y)\, x) \\ &\to_\beta \lambda f.\, \lambda x.\, f\,(f\, x) = \underline{2}. \end{aligned}$$

Solo con sustituciones se obtiene el sucesor.

Turing demostró que las funciones λ-definibles son exactamente las computables por sus máquinas [16]. La tesis de Church-Turing usa esa equivalencia como puente: lo que es calculable por un algoritmo lo calcula una máquina de Turing o, igualmente, el cálculo lambda [17]. La máquina de Turing modela el cómputo como cambios de estado en una cinta (estilo imperativo) y el cálculo lambda como reescritura de expresiones (estilo funcional); por eso un lenguaje funcional es Turing-completo aunque no tenga asignación.

c. Cinco lenguajes funcionales

1. **Haskell**: funcional puro, con evaluación perezosa.
2. **OCaml**: de la familia ML, con inferencia de tipos.
3. **Lisp / Scheme**: de los primeros lenguajes con raíces en el cálculo lambda.
4. **Erlang**: pensado para sistemas concurrentes y distribuidos.
5. **F#**: funcional sobre la plataforma .NET.

5. Z3

Z3 es un probador de teoremas de Microsoft Research que revisa la satisfacibilidad de fórmulas sobre una o más teorías [7]. Los archivos también corren en el playground [9], pero las salidas de abajo son de una ejecución local con `z3-solver` instalado en un entorno virtual dentro de `lab11`:

```
> .venv\bin\z3.exe --version
Z3 version 5.3.3 - 64 bit - build hashcode 430d19e697e2ec9bad79ff210d41b4a7a5fcfa3e
```

d. ¿Cómo se prueba con Z3 que una fórmula es siempre válida?

Z3 responde satisfacibilidad, así que se usa que F es válida si y solo si $\neg F$ es insatisfacible [8]. Se hace `(assert (not F))` y `(check-sat)`: `unsat` significa que F es válida; `sat` significa que hay una asignación que la hace falsa, y `(get-model)` la muestra como contraejemplo.

Caso válido, De Morgan: $\neg(a \wedge b) \leftrightarrow (\neg a \vee \neg b)$.

validez_demorgan.smt2

```
(declare-const a Bool)
(declare-const b Bool)
(define-fun demorgan () Bool
  (= (not (and a b)) (or (not a) (not b))))
(assert (not demorgan))
(check-sat)
```

```
> .venv\bin\z3.exe z3\validez_demorgan.smt2
unsat
```

La negación es insatisfacible, entonces la ley es válida.

Caso no válido, De Morgan mal aplicada: $\neg(a \wedge b) \leftrightarrow (\neg a \wedge \neg b)$.

no_validez.smt2

```
(declare-const a Bool)
(declare-const b Bool)
(define-fun f () Bool
  (= (not (and a b)) (and (not a) (not b))))
(assert (not f))
(check-sat)
(get-model)
```

```
> .venv\bin\z3.exe z3\no_validez.smt2
sat
(
  (define-fun b () Bool
    true)
  (define-fun a () Bool
    false)
  (define-fun f () Bool
    (= (not (and a b)) (and (not a) (not b))))
)
```

Con $a = \textit{false}$, $b = \textit{true}$ el lado izquierdo da *true* y el derecho *false*, así que la fórmula no es válida.

e. La función `mistery`

i. Resultado de ejecutar el código

mistery_11.smt2

```
(define-fun mistery ((x Int)) Bool
  (and
    (> x 1)
    (not (exists ((z Int) (y Int))
      (and (< y x) (< z x) (> y 1) (> z 1) (= x (* y z)))))))
(assert (mistery 11))
(check-sat)
```

```
> .venv\bin\z3.exe z3\mistery_11.smt2
sat
```

La fórmula no tiene variables libres (`x` vale 11; `y`, `z` están cuantificadas), así que `sat` significa que `(mistery 11)` es verdadera.

ii. Pruebas con $x = 1, \ldots, 20$ y qué hace la función

El script cambia solo el valor del `assert`, corre Z3 para cada x, imprime la tabla y la guarda en `salidas.txt`.

correr_mistery.py

```
import os
import subprocess
import sys
from pathlib import Path

base = Path(__file__).resolve().parent
plantilla = (base / "mistery_11.smt2").read_text(encoding="utf-8")
z3_exe = Path(sys.prefix) / "bin" / ("z3.exe" if os.name == "nt" else "z3")

def ejecutar(x):
    codigo = plantilla.replace("(assert (mistery 11))", f"(assert (mistery {x}))")
    r = subprocess.run([str(z3_exe), "-in"], input=codigo,
                       capture_output=True, text=True)
    return r.stdout.strip()

lineas = ["x  | resultado", "---+----------"]
for x in range(1, 21):
    lineas.append(f"{x:<2} | {ejecutar(x)}")
tabla = "\n".join(lineas)
print(tabla)
(base / "salidas.txt").write_text(tabla + "\n", encoding="utf-8")
```

Salida (`salidas.txt`):

```
x  | resultado
---+----------
```

```
1  | unsat
2  | sat
3  | sat
4  | unsat
5  | sat
6  | unsat
7  | sat
8  | unsat
9  | unsat
10 | unsat
11 | sat
12 | unsat
13 | sat
14 | unsat
15 | unsat
16 | unsat
17 | sat
18 | unsat
19 | sat
20 | unsat
```

Dan `sat` exactamente $2, 3, 5, 7, 11, 13, 17, 19$; el resto da `unsat` y no hubo ningún `unknown`. La fórmula dice

$$\mathit{mistery}(x) \;\equiv\; x > 1 \;\wedge\; \neg\exists y, z \in \mathbb{Z}\,(1 < y < x \;\wedge\; 1 < z < x \;\wedge\; x = y \cdot z),$$

es decir, $x > 1$ y x no es producto de dos enteros estrictamente entre 1 y x. Esa es la definición de primo. Por ejemplo, con $x = 1$ falla $x > 1$; con $x = 4$ existen $y = z = 2$, así que el $\exists$ es verdadero y da `unsat`; con $x = 11$ ningún par sirve y da `sat`. Entonces $\mathit{mistery}(x)$ es verdadero si y solo si x es primo.

La fórmula mezcla multiplicación entre variables y un cuantificador, el fragmento que la guía de Z3 marca como indecidible en general [5, 6]. Aquí funciona porque x es constante y las cotas $1 < y, z < x$ dejan un número finito de candidatos.

Referencias

[1] A. Gacek, "Protect Sensitive Data in the Cloud with Automated Reasoning: Zelkova", *AWS Security Blog*, 20 de junio de 2018 (actualizado el 31 de enero de 2025). `https://aws.amazon.com/blogs/security/protect-sensitive-data-in-the-cloud-with-automated-reasoning-zelkova/`

[2] Amazon Web Services, "AWS re:Invent 2016: Automated Formal Reasoning About AWS Systems (SEC401)", YouTube. `https://www.youtube.com/watch?v=U40bWY6oVtU`

[3] B. Cook, "Formal Reasoning About the Security of Amazon Web Services", en *Computer Aided Verification (CAV 2018)*, LNCS, vol. 10981. Springer, 2018. doi:10.1007/978-3-319-96145-3_3. `https://discovery.ucl.ac.uk/id/eprint/10075272/1/Cook2018_Chapter_FormalReasoningAboutTheSecurit.pdf`

[4] R. Nieuwenhuis, A. Oliveras y C. Tinelli, "Solving SAT and SAT Modulo Theories: From an Abstract Davis–Putnam–Logemann–Loveland Procedure to DPLL(T)", *Journal of the ACM*,

vol. 53, no. 6, 2006. https://homepage.cs.uiowa.edu/~tinelli/papers/NieOT-JACM-06.pdf

[5] Microsoft Research, "Arithmetic", *Z3 Guide*. https://microsoft.github.io/z3guide/docs/theories/Arithmetic

[6] Microsoft Research, "Quantifiers", *Z3 Guide*. https://microsoft.github.io/z3guide/docs/logic/Quantifiers

[7] Microsoft Research, "Introduction", *Z3 Guide*. https://microsoft.github.io/z3guide/docs/logic/intro

[8] Microsoft Research, "Propositional Logic: Satisfiability and Validity", *Z3 Guide*. https://microsoft.github.io/z3guide/docs/logic/propositional-logic

[9] Z3 Playground. https://jfmc.github.io/z3-play/

[10] P. Godefroid, M. Y. Levin y D. Molnar, "SAGE: Whitebox Fuzzing for Security Testing", *Communications of the ACM*, vol. 55, no. 3, 2012. doi:10.1145/2093548.2093564

[11] C. Cadar, D. Dunbar y D. Engler, "KLEE: Unassisted and Automatic Generation of High-Coverage Tests for Complex Systems Programs", en *8th USENIX Symposium on Operating Systems Design and Implementation (OSDI 08)*, San Diego, 2008. https://www.usenix.org/legacy/event/osdi08/tech/full_papers/cadar/cadar.pdf

[12] Microsoft, "Static Driver Verifier", *Microsoft Learn*. https://learn.microsoft.com/en-us/windows-hardware/drivers/devtest/static-driver-verifier

[13] Microsoft Research, "SLAM". https://www.microsoft.com/en-us/research/project/slam/

[14] T. Ball, E. Bounimova, V. Levin y L. de Moura, "Efficient evaluation of pointer predicates with Z3 SMT Solver in SLAM2", Microsoft Research, MSR-TR-2010-24, 2010.

[15] P. Hudak, "Conception, evolution, and application of functional programming languages", *ACM Computing Surveys*, vol. 21, no. 3, pp. 359–411, 1989.

[16] A. M. Turing, "Computability and λ-definability", *The Journal of Symbolic Logic*, vol. 2, no. 4, pp. 153–163, 1937.

[17] M. Sipser, *Introduction to the Theory of Computation*, 3.ª ed. Cengage Learning, 2013.

Se utilizó Claude para la revisión y corrección; el trabajo fue revisado y realizado por mí.